



IT GENIUS
THINK IT · BE GENIUS

DAY

3

HERMES AGENT

BUILD YOUR OWN AI EMPLOYEE

จากผู้ใช้ AI → สู่ผู้สร้าง AI Agent



Agent



Memory



Tools



Skills



Workflow



Multi-Agent



AI Employee

ลงมือทำจริง • สร้างใช้กับงานของคุณ



Memory



Long-term memory



Knowledge base



User context



Past tasks

AI Agent



Tools



Web search



API



File & Data



Automation



Workflow



AI

Productivity

Future Work

AI
BETTER
WORK

อ.เฟิร์ส วัชร น้อยศรีพันธ์

Tools, Skills & MCP

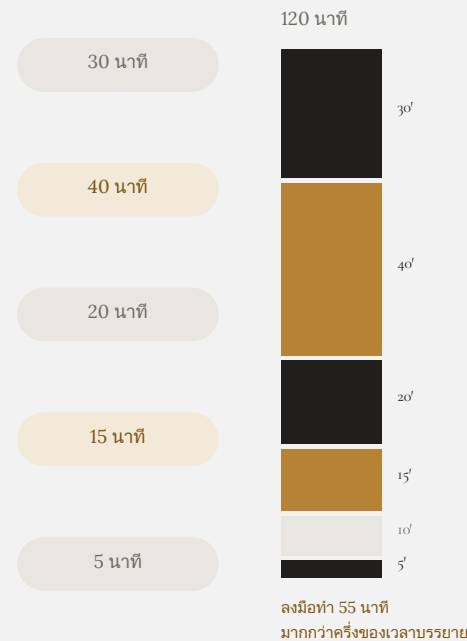
เพิ่มความสามารถให้ Agent — ให้ใช้เครื่องมือ ทำงานกับข้อมูล
และเชื่อมต่อระบบภายนอกได้



ยังต่อเครื่องมือ ยังทำงานแทนเราได้จริง

กำหนดการวันนี้

01	●	Tools & Tool Calling	20:00 – 20:30
02	●	Workshop 5 · Research & Computer Agent	20:30 – 21:10
03	●	Custom Skills	21:10 – 21:30
04	●	Workshop 6 · Build Your Own Skill	21:40 – 21:55
05	●	Mini Session · MCP	21:55 – 22:00



วันนี้อยู่ตรงไหนของหลักสูตร



Workshop Project เดิมของแต่ละคนถูกพัฒนาต่อทุกวัน — วันนี้เราเพิ่ม Tools, Skills และ MCP ให้ AI Employee ที่เริ่มไว้ตั้งแต่วันแรก

วันนี้คือจุดเปลี่ยน

สองวันแรก Agent “รู้เรื่องเรา” — ตั้งแต่วันนี้มันเริ่ม “ลงมือแทนเรา” และผลลัพธ์ออกมาเป็นไฟล์จริง

เป้าหมายของวันนี้

- 01 เข้าใจว่า Tool Calling ทำงานอย่างไร และ Agent เลือกเครื่องมือจากอะไร
ทำไมบางครั้งมันเลือกเครื่องมือผิด และเราแก้ที่ไหน
- 02 ให้ Agent ทำงานกับ File System, Terminal, Search และ API ได้จริง
ทำงานจริงหนึ่งขั้นให้จบ ตั้งแต่รับโจทย์จนได้ไฟล์
- 03 สร้าง Custom Skill ของตัวเองด้วย SKILL.md และนำกลับมาใช้ซ้ำ
เปลี่ยนงานที่ทำซ้ำของตัวเอง ให้เรียกใช้ได้ด้วยประโยคเดียว
- 04 เห็นภาพ MCP ในฐานะมาตรฐานเชื่อมต่อ Agent กับระบบภายนอก
เห็นภาพว่าวันที่ 4 กับ 5 จะต่ออะไรเข้ากับ Agent ได้บ้าง

DAY 3

Tools
Skills
& MCP

จบนี่ Agent ของเรา
จะมีเครื่องมือ และมีวิธีทำงาน
ที่เก็บไว้ใช้ซ้ำได้

ก่อนเริ่ม · เตรียมให้พร้อมใน 3 นาที

วันนี้ลงมือทำ 55 นาที จากทั้งหมด 120 — เวลาเตรียมเครื่องคือเวลาที่หายไปจาก Workshop

- | | | |
|----|--------------------------------|--|
| 01 | เปิด Agent ที่ใช้เครื่องมือได้ | Hermes / Claude Code / เครื่องมือที่ทีมใช้ — ขอให้เป็นตัวที่อ่านเขียนไฟล์ได้ |
| 02 | สร้างโฟลเดอร์งานของตัวเอง | ตั้งชื่อว่า day3-workshop แล้วให้ Agent ทำงานอยู่ที่นี่เท่านั้น |
| 03 | เปิด Memory จากวันที่ 2 ไว้ | User Memory และ Project Memory ที่เขียนไว้ วันนี้จะถูกใช้ต่อ |
| 04 | เตรียมงานจริงหนึ่งชิ้นในใจ | งานที่ทำซ้ำทุกสัปดาห์ — Workshop 6 จะเปลี่ยนงานนั้นให้เป็น Skill |

ติดตรงไหนให้พิมพ์ในแชทห้องได้เลย — ระหว่าง Workshop มีผู้ช่วยคอยตอบ ไม่ต้องรอนจนจบ

OI

Tools & Tool Calling

Session 5 · 20:00 – 20:30

จาก “ตอบได้” เป็น “ทำได้”

ไม่มี TOOL

Model ที่รู้แต่ไม่ทำ

คำตอบทั้งหมดมาจากสิ่งที่อยู่ใน Context เท่านั้น อ่านไฟล์จริงไม่ได้ ค้นข้อมูลใหม่ไม่ได้ และผลลัพธ์จบลงที่ข้อความในหน้าจอ

มี TOOL

Agent ที่เปลี่ยนสภาพแวดล้อม

Agent เรียกเครื่องมือเพื่ออ่านไฟล์ รันคำสั่ง ค้นข้อมูล และเขียนผลลัพธ์กลับออกมาเป็นไฟล์จริง สิ่งที่เกิดขึ้นตรวจสอบได้จากงานที่ทำเสร็จ

ความต่างที่ตรวจสอบได้

ผลลัพธ์จบที่ไหน

ข้อความในหน้าจอ

ไฟล์ในโพลเดอร์ของเรา

ตรวจสอบย้อนหลังได้ไหม

ได้แค่อ่านคำตอบซ้ำ

ตามได้ว่าอ่านอะไร สิ่งอะไร ได้อะไรกลับ

เมื่อข้อมูลเปลี่ยน

ต้องเล่าให้ฟังใหม่ทั้งหมด

สั่งให้ไปอ่านใหม่ แล้วมันทำเอง

คำถามเดียวกัน · Agent สองแบบ

ความต่างไม่ได้อยู่ที่โมเดลฉลาดกว่ากัน แต่อยู่ที่ตัวหนึ่งไปดูของจริงได้

“ยอดขายเดือนที่แล้วของสาขาไหนตกมากที่สุด”

ไม่มี TOOL

ตอบจากสิ่งที่อยู่ใน Context

Agent ผมไม่มีข้อมูลยอดขายของบริษัทครับ

Agent ถ้าคุณวางตัวเลขมาให้ ผมช่วยวิเคราะห์ได้

จบที่ข้อความ — เราต้องไปหาข้อมูลเอง

มี TOOL

ไปอ่านของจริงก่อนตอบ

Agent ls data/ → พบ sales_2569_08.csv

Agent อ่านไฟล์ 1,204 แถว · รวมยอดตามสาขา

Agent สาขาโรงสิต -18.4% เขียนผลลงใน drop.md

จบที่ไฟล์จริง — ตรวจสอบได้ทุกชั้น

โมเดลตัวเดียวกันทั้งสองฝั่ง — สิ่งที่เพิ่มเข้ามาคือสิทธิ์ในการอ่านไฟล์ และหน้าที่ต้องอ่านผลก่อนสรุป

Hermes Agent Live Online · Dav 3

Tool Calling ทำงานอย่างไร



Agent เลือกเครื่องมือจากคำอธิบายของ Tool ประกอบกับบริบทของงาน — ขั้นที่ 4 คือหัวใจ เพราะ Agent ต้องอ่านผลลัพธ์จริงก่อนตัดสินใจขั้นถัดไป ไม่ใช่เดาว่าคำสั่งสำเร็จ

ลูปนี้วนได้หลายรอบในงานเดียว

งานจริงหนึ่งชิ้นอาจวนสืบกกว่ารอบ — แต่ละรอบ Agent ตัดสินใจใหม่จากผลลัพธ์ที่เพิ่งได้
ถ้ามันข้ามขั้นที่ 4 ไปตอบเลย นั่นคือจุดที่คำตอบเริ่มเป็นการเดาที่เขียนสวย

Agent มองเห็น Tool เป็นอย่างไร

Agent ไม่เห็นโค้ดของเครื่องมือ — เห็นแค่ป้ายชื่อสามบรรทัดนี้ แล้วตัดสินใจจากมัน

NAME

ชื่อที่ใช้เรียก

`read_file`

สั้น ตรงตัว สื่อว่าทำอะไร — ชื่อ
คลุมเครือทำให้ Agent เรียกผิดตัว

DESCRIPTION

ใช้เมื่อไร

“อ่านเนื้อหาไฟล์จาก path ที่
ระบุ ใช้เมื่อ
ต้องดูข้อมูลจริงก่อนตอบ”

บรรทัดที่สำคัญที่สุด — Agent เลือก
เครื่องมือจากประโยคนี้เป็นหลัก

PARAMETERS

ต้องการอะไร

`path: string` (บังคับ)
`lines: number` (ไม่บังคับ)

ถ้าไม่ระบุอะไรบังคับ Agent จะ
เดา แล้วเรียกพลาตรอบแรกเสมอ

เขียน Tool ให้ Agent ใช้เป็น = เขียน Description ให้คนใหม่ในทีมอ่านแล้วรู้ว่าควรหยิบตอนไหน

เดินตามรูปจริง หนึ่งงาน

โจทย์ · “สรุปว่าไฟล์ log เมื่อคืนมี error ก็ครั้ง และเป็นเรื่องอะไร”

รอบ	Agent คิดอะไร	เรียกเครื่องมือ	ได้อะไรกลับมา
01	ยังไม่รู้ว่าไฟล์อยู่ไหน ต้องดูก่อน	<code>list_dir("logs/")</code>	พบ app-0913.log ขนาด 4.2 MB
02	ไฟล์ใหญ่เกินกว่าจะอ่านทั้งก้อน	<code>grep("ERROR", "app-0913.log")</code>	เจอ 37 บรรทัดที่ตรงเงื่อนไข
03	37 บรรทัดพอไหว อ่านของจริง	<code>read_lines</code> (ทั้ง 37 บรรทัด)	ข้อความ error เต็ม ๆ พร้อมเวลา
04	จัดกลุ่มดูว่าซ้ำเรื่องเดียวกันไหม	– คิดเอง ไม่ต้องใช้ tool –	3 กลุ่ม · timeout 29 · auth 6 · อื่น 2
05	ส่งเป็นไฟล์ให้เปิดอ่านย้อนได้	<code>write_file("error-summary.md")</code>	ไฟล์จริงในโฟลเดอร์ พร้อมเวลาและตัวอย่าง

สังเกตรอบที่ 2 — Agent เปลี่ยนวิธีเพราะเห็นว่าไฟล์ใหญ่เกินไป นั่นคือการอ่านผลลัพธ์จริง ไม่ใช่เดา

เครื่องมือสีกลุ่มที่ใช้วันนี้

File System

อ่าน เขียน และจัดโครงสร้างไฟล์ใน Workspace

- อ่าน sales.csv 1,204 แถว
- สร้าง report.md
- ดูว่าในโฟลเดอร์มีอะไรบ้าง

Terminal

รันคำสั่งจริงและอ่าน Output กลับมาใช้ต่อ

- grep หา ERROR ใน log
- รันเทสแล้วอ่านผล
- แปลงไฟล์ด้วยคำสั่งเดียว

Search

ค้นข้อมูลใหม่ที่ไม่ได้อยู่ใน Context ของ Model

- ราคาคู่แข่งสัปดาห์นี้
- ประกาศที่เพิ่งออกเมื่อวาน
- เอกสารเวอร์ชันล่าสุด

API

ดึงและส่งข้อมูลกับระบบอื่นตามสิทธิ์ที่ได้รับ

- ดึง Issue จากระบบทีม
- อ่านตัวเลขจากฐานข้อมูล
- ส่งสรุปเข้าห้องแชท

งานแบบนี้ ใช้เครื่องมือไหน

เครื่องมือผิดประเภททำให้งานช้าและ Context บวม มากกว่าทำให้ผิด

อยากรู้ว่ามีไฟล์อะไรบ้าง

File System

list / glob — อย่าเพิ่งอ่านเนื้อหา

หาคำหรือรูปแบบในไฟล์จำนวนมาก

Terminal

grep เร็วกว่าให้ Agent ไล่อ่านทีละไฟล์มาก

ต้องเข้าใจเนื้อหาไฟล์จริง ๆ

File System

อ่านเฉพาะช่วงที่ต้องใช้ ไม่ใช่ทั้งไฟล์

ข้อมูลใหม่กว่าที่โมเดลรู้

Search

ข่าว ราคา ประกาศ เอกสารที่เพิ่งออก

ต้องได้ตัวเลขจากระบบของเรา

API

ดึงตามสิทธิ์ที่ได้รับ อย่า copy ข้อมูลมาวางในแชท

ต้องรัน ต้องทดสอบ ต้องแปลงไฟล์

Terminal

สั่งแล้วอ่าน output กลับมาตัดสินใจต่อ

หลักง่าย ๆ — แคมก่อนค่อยลิก · หาให้เจอก่อนแล้วค่อยอ่าน · อ่านเท่าที่ต้องใช้

สื่ออาการที่เจอบ่อย และทางแก้

เกือบทุกครั้งที่ Agent “ทำไม่ได้” คือหนึ่งในสี่ข้อนี้ ไม่ใช่โมเดลไม่เก่ง

อาการ

Agent ตอบว่าทำเสร็จแล้ว แต่ไม่มีไฟล์

ทางแก้

สั่งให้ยืนยันด้วยการ list โฟลเดอร์ แล้วอ่านไฟล์กลับมาให้ดู

อาการ

เรียกเครื่องมือเดิมวนซ้ำไม่จบ

ทางแก้

หยุดแล้วถามว่า “ตอนนี้ติดอะไร” — ให้มันสรุปสถานะก่อนไปต่อ

อาการ

ขอสิทธิ์ทุกคำสั่งจนทำงานไม่สิ้น

ทางแก้

อนุญาตล่วงหน้าเฉพาะคำสั่งอ่านอย่างเดียว เก็บการถามไว้ให้คำสั่งที่เขียนทับ

อาการ

ผลลัพธ์ยาวจน Context เต็มแล้วลืมโจทย์

ทางแก้

ให้กรองก่อนอ่าน เขียนผลลงไฟล์ แล้วอ้างถึงไฟล์แทนการวางเนื้อหา

เก็บวิธีแก้ที่ได้ผลไว้ — พอมันซ้ำรอบที่สาม นั่นคือสัญญาณว่าควรเขียนเป็น Skill

Permission และ Security

ยิ่ง Agent ทำได้มาก ความเสี่ยงยิ่งย้ายจากคำตอบผิด ไปเป็นการกระทำผิด — ก่อนเข้า Workshop จึงต้องตกลงขอบเขตให้ชัด

O1	กำหนด Workspace ให้ชัด และให้ Agent ทำงานอยู่ในขอบเขตนั้น	โฟลเดอร์เดียว จบในนั้น ลดความเสียหายเหลือเท่าที่อยู่ในนั้น
O2	แยกคำสั่งที่ทำตัวเอง ออกจากคำสั่งที่ต้องขออนุญาตก่อน	อ่านได้เอง แต่เขียนทับต้องถาม เส้นแบ่งอยู่ที่ “ย้อนกลับได้ไหม”
O3	ไม่ใส่ Credential จริงลงใน Context หรือไฟล์ที่ Agent อ่านได้	ให้ชื่อ Environment Variable ไป อย่าให้ ค่าจริง
O4	ให้ Agent สรุปลสิ่งที่ทำไปทุกครั้ง เพื่อให้ตรวจสอบย้อนหลังได้	สรุปหนึ่งย่อหน้าทำงาน ทำให้ตรวจสอบ ได้โดยไม่ต้องไล่อ่านทั้งหมด

ตั้งขอบเขตครั้งเดียวตอนเริ่มงาน ใช้เวลา 30 วินาที — ถูกกว่าการตามเก็บทีหลังมาก

แบ่งสิทธิ์เป็นสามระดับ

ไม่ใช่ “ให้ทุกอย่าง” หรือ “ถามทุกครั้ง” — แบ่งตามว่าถ้าพลาดแล้วย้อนกลับได้ไหม

ALLOW

ทำได้เอง

- อ่านไฟล์ในโฟลเดอร์งาน
- ค้นหาข้อความ · ดูรายชื่อไฟล์
- รันคำสั่งที่ไม่แก้ไขของ
- ค้นเว็บสาธารณะ

พลาดแล้วไม่เสียหาย · ย้อนได้ทันที

ASK

ถามก่อน

- เขียนทับหรือลบไฟล์
- ติดตั้งอะไรลงเครื่อง
- เรียก API ที่เปลี่ยนข้อมูล
- ส่งอะไรออกนอกเครื่อง

ย้อนได้ แต่ต้องรู้ตัวก่อนว่ากำลังจะเกิดอะไร

NEVER

ห้ามเด็ดขาด

- แตะ Production โดยตรง
- ใช้ Credential จริงในแซท
- ส่งข้อมูลลูกค้าออกนอกระบบ
- ลบทั้งโฟลเดอร์

พลาดครั้งเดียวย้อนไม่ได้ · ต้องมีคนทำเอง

ตกลงสามระดับนี้กับตัวเองก่อนเข้า Workshop — แล้วบอก Agent ตรง ๆ ในบรรทัดแรกของงาน

02

Research & Computer Agent

Workshop 5 · 20:30 – 21:10

โจทย์ของ Workshop 5 · เลือกหนึ่งเส้น

ทุกเส้นใช้เครื่องมือชุดเดียวกัน และจบที่ไฟล์ Report หนึ่งไฟล์เหมือนกัน

เส้น A

คู่แข่งและตลาด

เลือกคู่แข่งหรือสินค้าหนึ่งตัวที่เกี่ยวกับงานเรา

- 1 ค้นข้อมูลล่าสุด 5 แหล่ง
- 2 สรุปจุดแข็ง-จุดอ่อน
- 3 เสนอ 3 ข้อที่เราทำได้

เส้น B

ข้อมูลในเครื่องเรา

เอาไฟล์งานจริงที่มีอยู่แล้ว — csv, log, เอกสาร

- 1 ให้ Agent สืบรวจโครงสร้าง
ก่อน
- 2 ถ้ามีคำถามที่ตอบได้จากข้อมูล
- 3 สรุปพร้อมตัวเลขอ้างอิง

เส้น C

งานประจำที่ทำซ้ำ

งานที่เราทำเองอยู่ทุกสัปดาห์ ให้ Agent ทำแทนหนึ่งรอบ

- 1 อธิบายขั้นตอนที่เราทำ
- 2 ให้ Agent ทำแล้วเทียบผล
- 3 บันทึกจุดที่ต้องแก้

เลือกภายใน 2 นาทีแล้วเริ่มเลย — เส้นไหนก็ได้ ขอให้เป็นเรื่องที่เราสนใจจริง งานจะเดินเร็วกว่ามาก

Prompt ตั้งต้น · คัดลอกไปใช้ได้เลย

เปลี่ยนเฉพาะข้อความในวงเล็บ ที่เหลือปล่อยไว้ตามนี้

ทำงานอยู่ในโฟลเดอร์ `day3-workshop` เท่านั้น

โจทย์: [เขียนโจทย์ของเล่นที่เลือก 1-2 บรรทัด]

ขอให้ทำตามนี้

1. สรุปโจทย์เป็นแผนย่อยให้ผมอ่านก่อน แล้วรอผมตอบ ok
2. ค้นและอ่านข้อมูลจริง อย่าตอบจากความจำ
3. บอกทุกครั้งว่าใช้เครื่องมืออะไร และได้อะไรกลับมา
4. เขียนผลลงไฟล์ `report.md` ในโฟลเดอร์นี้
5. ปิดท้ายด้วยรายการแหล่งที่มา และสิ่งที่ยังไม่แน่ใจ

ถ้าจะลบหรือเขียนทับไฟล์เดิม ให้ถามผมก่อนเสมอ

Prompt ที่ดีคือสัญญาว่าจะส่งอะไรกลับ ไม่ใช่คำอธิบายว่าอยากได้อะไรสวย ๆ

Hermes Agent Live Online · Dav 3

ทำไมต้องมีบรรทัดพวกนี้

ขอบเขต

กันไม่ให้ Agent เดินออกไปแตะไฟล์อื่นในเครื่อง

รอ ok

ได้เห็นแผนก่อนเสียเวลา 20 นาทีไปกับแผนที่ผิด

ห้ามตอบจากความจำ

บังคับให้เรียกเครื่องมือจริง ไม่ใช่เดาแล้วเขียนสวย

รายงานเครื่องมือ

ทำให้เราตรวจสอบได้ว่าข้อสรุปมาจากไหน

เขียนลงไฟล์

ผลงานอยู่นอกเซต เอาไปใช้ต่อและส่งต่อได้

บอกสิ่งที่ไม่แน่ใจ

ลดการเดาเด๋มช่องว่างด้วยข้อความที่ฟังดูดี

ลำดับงานของ Workshop 5



หนึ่งโจทย์ เดินครบทุกขั้น โดยให้ Agent ใช้ Search, File System และ Terminal ในงานเดียวกัน — ปลายทางคือไฟล์ Report ที่เปิดอ่านได้จริง ไม่ใช่ข้อความในแชท

เวลาที่ให้ในแต่ละขั้น

รับโจทย์

3'

ค้นและอ่าน

7'

วิเคราะห์

5'

สร้างไฟล์

8'

ตรวจสอบจริง

5'

บันทึกวิธีทำ

4'

เดินทีละขั้น · ขั้นที่ I ถึง 3

O1	รับโจทย์ · 3 นาที สิ่งที่เราพิมพ์	วาง Prompt ตั้งต้น แล้วอ่านแผนที่ Agent สรุปกลับมา ต้องเห็น · แผนเป็นข้อ ๆ ที่เราอ่านแล้วเห็นภาพ ไม่ใช่ย่อหน้ากว้าง ๆ	ถ้าแผนไม่ตรง อย่าปล่อยผ่าน — แก่ตรงนี้ถูกกว่าแก้ตอนจบ
O2	ค้นและอ่าน · 7 นาที สิ่งที่เราพิมพ์	ให้ Agent ใช้ Search หรืออ่านไฟล์จริง แล้วรายงานแหล่งที่มา ต้องเห็น · รายการแหล่งข้อมูลที่คลิกเปิดได้ หรือชื่อไฟล์กับบรรทัดจริง	เจอสรุปที่ไม่มีที่มา ให้ถามกลับว่า “อันนี้มาจากไหน”
O3	วิเคราะห์ · 5 นาที สิ่งที่เราพิมพ์	ให้เทียบข้อมูลที่ได้ แล้วตัดสิ่งที่ไม่เกี่ยวกับโจทย์ออก ต้องเห็น · ข้อสรุปที่มีเหตุผลรองรับ และบอกด้วยว่าอะไรยังไม่แน่ใจ	ถ้าทุกอย่างฟังดูดีไปหมด แปลว่ายังไม่ได้อะไรเลย

จับเวลาจริง — ขั้นละไม่เกินนาที ถ้าเกินให้ตัดขอบเขตโจทย์ลง อย่ายืดเวลา

เดินทีละขั้น · ขั้นที่ 4 ถึง 6

04

สร้างไฟล์ · 8 นาที

สิ่งที่เราพิมพ์

สั่งให้เขียน report.md แล้วให้เปิดไฟล์นั้นอ่านกลับ
มาให้เราดู

ต้องเห็น · ไฟล์จริงในโฟลเดอร์ ไม่ใช่ข้อความยาว ๆ ในแชท

ให้มัน list โฟลเดอร์ยืนยันอีกรอบ — กันเคส “บอก
ว่าเขียนแล้ว”

05

ตรวจสอบจริง · 5 นาที

สิ่งที่เราพิมพ์

เปิดไฟล์ด้วยตาเราเอง แล้วสุ่มตรวจสอบข้ออ้าง 2 ข้อว่า
ตรงกับแหล่งใหม่

ต้องเห็น · ทุกข้อสรุปสำคัญ ตามกลับไปหาที่มาได้

ขั้นนี้คนตรวจ ไม่ใช่ Agent ตรวจสอบตัวเอง

06

บันทึกวิธีทำ · 4 นาที

สิ่งที่เราพิมพ์

ถามว่า “สรุปขั้นตอนที่เพิ่งทำเป็นรายการสั้น ๆ ให้
หน่อย”

ต้องเห็น · ลำดับขั้นที่เอาไปทำซ้ำได้โดยไม่ต้องนึกเอง

ไฟล์นี้แหละคือวัตถุดิบของ Workshop 6

เหลืออีก 5 นาทีสุดท้ายไว้แฉร์ผล — เตรียมตอบสั้น ๆ ว่า “Agent ทำอะไรที่เราไม่คาดคิด”

สิ่งที่ต้องได้จาก Workshop 5

ตารางนี้คือเกณฑ์ตรวจ — ช่องขาวคือสิ่งที่ต้องชี้ให้ดูได้ ไม่ใช่สิ่งที่รู้สึกว่าได้ทำแล้ว

ขั้นตอน	สิ่งที่ AGENT ทำ	สิ่งที่ต้องเห็น
รับโจทย์	สรุปโจทย์เป็นแผนย่อย	แผนที่อ่านเข้าใจก่อนเริ่มทำ
ค้นและอ่าน	ใช้ Search แล้วอ่านแหล่งที่มา	รายการแหล่งข้อมูลที่น่าเชื่อถือได้
วิเคราะห์	เทียบข้อมูลและตัดสิ่งที่ไม่เกี่ยว	ข้อสรุปที่มีเหตุผลรองรับ
สร้างไฟล์	เขียน Report ลง Workspace	ไฟล์จริงในโฟลเดอร์ของตัวเอง
บันทึกผล	เก็บผลลัพธ์และวิธีทำไว้ใช้ซ้ำ	บันทึกที่ต่อยอดเป็น Skill ได้

ทั้งห้าแถวนี้ใช้เวลารวม 32 นาที — ที่เหลือเผื่อไว้ให้ตรวจและแก้

เช็ีกก่อนบอกว่าเสร็จ

ดีถ้าให้ครบทั้งหกข้อ — ขาดข้อเดียวแปลว่ายังเป็นแค่ข้อความที่ฟังดูดี

- ☐ มีไฟล์ report.md อยู่จริงในโฟลเดอร์ และเราเปิดอ่านแล้ว
- ☐ ทุกข้อสรุปสำคัญ มีที่มาระบุไว้ — ลิงก์ ชื่อไฟล์ หรือเลขบรรทัด
- ☐ เราสรุปตรวจย้อนกลับแล้วอย่างน้อยสองข้อ และมันตรง
- ☐ มีหัวข้อ “สิ่งที่ยังไม่แน่ใจ” และมันไม่ว่างเปล่า
- ☐ Agent ไม่ได้แตะไฟล์นอกโฟลเดอร์งานเลยสักครั้ง
- ☐ มีบันทึกขั้นตอนที่ทำ พร้อมเอาไปทำซ้ำพรุ่งนี้ได้

ข้อที่คนพลาดมากที่สุดคือข้อ 3

Agent เขียนได้เนียนพอที่จะทำให้เราขี้เกียจตรวจ — การสรุปตรวจสอบข้อใช้เวลา 1 นาที และเปลี่ยนทุกอย่าง

ติดตรงนี้ . ทำแบบนี้

เจอบ่อยที่สุดสี่แบบระหว่าง Workshop — แก้ได้ในไม่กี่วินาที

! Agent ถามกลับเยอะจนไม่ได้เริ่มสักที

! มันเขียนสรุปสวายแต่ไม่ได้เปิดไฟล์จริงเลย

! ค้นเว็บแล้วได้แต่ของทั่วไป ไม่ตรงโจทย์

! งานยาวจนมันเริ่มลืมโจทย์ตั้งต้น

→ ตอบรวดเดียวทุกข้อแล้วปิดท้ายว่า “ที่เหลือดัดสินใจเองได้ บอกผมทีหลัง”

→ “ก่อนสรุป ให้ list โพลเดอร์และอ่านไฟล์จริงก่อน แล้วบอกว่าอ่านอะไรไปบ้าง”

→ ระบุขอบเขตให้แคบ — ช่วงเวลา ประเทศ ชื่อสินค้า แล้วสั่งให้ค้นซ้ำด้วยคำที่แคบลง

→ “สรุปสถานะตอนนี้ 5 บรรทัด” แล้วเปิด Session ใหม่ด้วยสรุปนั้น + ไฟล์ที่มีอยู่

ทุกข้อคือเรื่องเดียวกัน — เรากำลังบอกให้มัน “ไปดูของจริง” แทนที่จะ “เขียนให้ฟังดูดี”

เสร็จก่อนเวลา · ลองต่ออีกสามข้อ

ทั้งสามข้อคือสิ่งที่จะได้ใช้จริงในวันที่ 4 ตอนทำ Multi-Agent

ให้มันตรวจงานตัวเอง

จะเห็นว่ามันหาจุดอ่อนของตัวเองเจอ ถ้าเราสั่งให้หา

“อ่าน `report.md` ที่เพิ่งเขียน แล้วหาข้ออ้างที่ยังไม่มีหลักฐานรองรับ”

ให้มันทำซ้ำกับอีกหัวข้อ

รอบสองจะเร็วกว่ารอบแรกมาก — นี่คือเหตุผลที่ Skill มีค่า

“ใช้ขั้นตอนเดิมทั้งหมด เปลี่ยนหัวข้อเป็น ____ แล้วเขียนเป็นไฟล์ที่สอง”

ให้มันเปรียบเทียบสองไฟล์

งานเปรียบเทียบคือจุดที่ Agent ชนะการอ่านเองชัดที่สุด

“เทียบ `report.md` กับ `report-2.md` แล้วบอกว่าข้อสรุปขัดกันตรงไหน”

ยังไม่เสร็จก็ไม่ใช่ไร — เอาเท่าที่ได้ไปต่อใน Workshop 6 ได้เลย

Hermes Agent Live Online · Day 3

03

Custom Skills

Session 6 · 21:10 – 21:30

Skill ต่างจาก Prompt อย่างไร

PROMPT

คำสั่งครั้งเดียว

อยู่กับ Session นั้น เมื่อเปลี่ยนงานหรือเปิดใหม่ ต้องอธิบายวิธีทำงานซ้ำอีกครั้ง และคุณภาพขึ้นกับว่าจำได้ว่าเคยสั่งอย่างไร

SKILL

วิธีทำงานที่เก็บไว้

เป็น Procedural Memory ที่ Agent หยิบมาใช้เมื่อเจองานประเภทเดิม ทีมใช้ร่วมกันได้ แก่ทีเดียวแล้วดีขึ้นทุกครั้งที่เรียกใช้

เทียบให้เห็นชัด

อยู่ได้นานแค่ไหน

จบเมื่อปิด Session

อยู่ถาวรจนกว่าจะแก้

ใครใช้ได้

คนที่จำได้ว่าเคยสั่งอย่างไร

ทุกคนในทีมที่เรียกชื่อมันได้

แก้ให้ดีขึ้นตรงไหน

แก้ทีละครั้ง แล้วก็หายไป

แก้ที่ไฟล์เดียว ดีขึ้นทุกครั้งที่เรียก

งานแบบไหนควรทำเป็น Skill

ไม่ใช่ทุกอย่างควรเป็น Skill — เขียนเยอะเกินไปแล้วไม่มีใครใช้ ก็เปล่าประโยชน์

สัญญาณว่าควรเขียน

- ✓ ทำซ้ำอย่างน้อยสัปดาห์ละครั้ง
- ✓ มีลำดับขั้นที่ค่อนข้างตายตัว
- ✓ คนอื่นในทีมก็ต้องทำงานนี้เหมือนกัน
- ✓ ผลลัพธ์ต้องหน้าตาเหมือนเดิมทุกครั้ง
- ✓ เราเคยอธิบายวิธีทำให้ Agent ฟังมาแล้วหลายรอบ

สัญญาณว่ายังไม่ต้อง

- ✗ ทำครั้งเดียวแล้วจบ
- ✗ ทุกครั้งวิธีทำไม่เหมือนกันเลย
- ✗ ยังไม่รู้ว่าผลลัพธ์ที่ดีหน้าตาเป็นยังไง
- ✗ เป็นแค่คำสั่งสั้น ๆ ที่พิมพ์ใหม่ก็เร็วพอ
- ✗ ขั้นตอนยังเปลี่ยนทุกสัปดาห์ — รอให้นิ่งก่อน

เริ่มจากงานเดียวที่น่าเบื่อที่สุดวันสัปดาห์ — นั่นคือ Skill แรกที่คุ้มที่สุดเสมอ

โครงสร้างของ SKILL.md

INPUT

รับอะไรเข้ามา

ระบุข้อมูลที่ต้องมีก่อนเริ่ม และรูปแบบที่ Agent คาดหวังจากผู้ใช้

คำถามที่ต้องตอบให้ได้

ต้องมีอะไรอยู่แล้ว ถึงจะเริ่มได้ · ถ้าไม่มีจะทำอย่างไร

PROCESS

ทำตามขั้นตอนไหน

ลำดับการทำงาน เครื่องมือที่ใช้ในแต่ละขั้น และเงื่อนไขที่ต้องหยุดถาม

คำถามที่ต้องตอบให้ได้

ทำอะไรก่อนหลัง · ตรงไหนต้องหยุดถาม · อะไรที่ห้ามทำ

OUTPUT

ส่งอะไรกลับ

รูปแบบผลลัพธ์ ที่เก็บไฟล์ และเกณฑ์ที่ถือว่างานนี้เสร็จแล้ว

คำถามที่ต้องตอบให้ได้

ไฟล์ชื่ออะไร อยู่ที่ไหน · แบบไหนเรียกว่าเสร็จ

เขียนให้คนอ่านรู้เรื่องก่อน แล้ว Agent จะทำตามได้ — Skill ที่ดีคือขั้นตอนที่ชัดเจนพอให้คนใหม่ในทีมทำงานตามได้เหมือนกัน

ลำดับการเขียนที่เร็วที่สุด

1 เขียน Output ก่อน

รู้ปลายทางแล้วขั้นตอนจะสั้นลงเอง

2 แล้วค่อยเขียน Process

ย้อนจากปลายทางว่าต้องผ่านอะไรบ้าง

3 ปิดท้ายด้วย Input

เหลือแค่ว่าต้องมีอะไรถึงจะเริ่มได้

SKILL.md ตัวจริง . อ่านทีละบรรทัด

```
---
name: weekly-sales-report
description: สรุปยอดขายรายสัปดาห์จากไฟล์ csv
             ใช้เมื่อผู้ใช้ขอ "สรุปยอดสัปดาห์นี้"
---

## Input
- ไฟล์ csv ในโฟลเดอร์ data/ ชื่อขึ้นต้นด้วย sales-
- ถ้ามีหลายไฟล์ ให้ถามก่อนว่าจะใช้ไฟล์ไหน

## Process
1. อ่านไฟล์ แล้วบอกจำนวนแถวกับช่วงวันที่
2. รวมนายตามสาขา และเทียบกับสัปดาห์ก่อน
3. ถ้าสาขาไหนตกเกิน 10% ให้ดึงรายการสินค้า
   ที่ตกมากที่สุด 5 อันดับของสาขานั้นมาด้วย
4. ห้ามเอาตัวเลขที่ไม่มีในไฟล์

## Output
- ไฟล์ reports/YYYY-WW.md
- ขึ้นต้นด้วยสรุป 3 บรรทัด แล้วค่อยตาราง
- ปิดท้ายด้วยสิ่งที่ข้อมูลยังตอบไม่ได้
```

ทั้งไฟล์ยาวไม่เกินหนึ่งหน้าจอ — ยาวกว่านั้นแปลว่ากำลังเขียนคู่มือ ไม่ใช่ Skill
Hermes Agent Live Online · Dav 3

บรรทัด name

ชื่อที่เรียกใช้ ใช้ตัวพิมพ์เล็กและขีดกลาง

บรรทัด description

สำคัญที่สุด — Agent อ่านบรรทัดนี้เพื่อตัดสินใจจะหยิบ Skill นี้ไหม
ต้องมีคำที่ผู้ใช้พูดจริง

Input

ระบุว่าต้องมีอะไรก่อนเริ่ม และถ้าไม่ครบให้ถาม ไม่ใช่เดา

Process ข้อ 3

เงื่อนไขแบบ “ถ้า...ให้...” คือจุดที่ Skill เก่งกว่า Prompt ธรรมดา

Process ข้อ 4

ข้อห้ามหนึ่งบรรทัด กันความเสียหายได้มากกว่าคำอธิบายสืบบรรทัด

Output

บอกที่เก็บและรูปแบบ เพื่อให้ผลลัพธ์เหมือนเดิมทุกครั้ง

เขียนอย่างไรให้ Agent ทำตามได้

กว้างจนทำตามไม่ได้

ชัดเจนที่จะตรวจได้

“สรุปให้ดี ๆ”



ขึ้นต้นด้วยสรุป 3 บรรทัด แล้วตารางตามสาขา ปิดท้ายด้วยข้อจำกัด

“ตรวจสอบข้อมูลก่อน”



บอกจำนวนแถวและช่วงวันที่ที่อ่านได้ ก่อนเริ่มคำนวณทุกครั้ง

“ถ้าผิดปกติให้บอก”



ถ้ายอดสาขาไหนตกเกิน 10% ให้ดึง 5 อันดับสินค้าที่ตกมากที่สุดมาด้วย

“อย่ามั่ว”



ห้ามใส่ตัวเลขที่ไม่มีในไฟล์ · ถ้าข้อมูลขาด ให้เขียนว่า “ไม่มีข้อมูล”

“เก็บไว้ให้เรียบร้อย”



เขียนลง reports/YYYY-WW.md · ถ้ามีไฟล์เดิมอยู่แล้ว ให้ถามก่อนเขียนทับ

กฎเดี่ยวที่ใช้ตัดสิน

อ่านแล้วบอกได้ไหมว่า Agent “ทำตาม” หรือ “ไม่ทำตาม” — ถ้าเลียงกันได้ แปลว่ายังไม่ชัดเจน

Skill จะถูกเรียกใช้ก็ต่อเมื่อ Agent หามันเจอ

เขียน Skill ดีแค่ไหน ถ้า description ไม่มีค่าที่ผู้ใช้พูดจริง มันก็ไม่ถูกหยิบขึ้นมา

ผู้ใช้พิมพ์จริง

description ต้องมีคำพวกนี้

“ขอสรุปยอดขายสัปดาห์นี้หน่อย”



สรุปยอดขาย · รายสัปดาห์ · weekly sales

“ทำรายงานให้ที่ขายอาทิตย์ที่แล้ว”



รายงาน · ยอดขาย · สัปดาห์ที่แล้ว

“sales report สัปดาห์นี้”



sales report · csv · ยอดตามสาขา

วิธีทดสอบที่เร็วที่สุด

ปิดแล้วเปิดใหม่ พิมพ์ประโยคที่จะพูดจริงในวันทำงาน — ถ้ามันไม่หยิบ Skill ขึ้นมาเอง ให้ไปแก้ description ไม่ใช่แก้ขั้นตอน

04

Build Your Own Skill

Workshop 6 · 21:40 – 21:55

Workshop 6 · Build Your Own Skill

21:40 – 21:55 น. · เลือกงานที่ตัวเองทำซ้ำอยู่แล้วหนึ่งงาน แล้วเปลี่ยนให้เป็น Skill ที่เรียกใช้ได้

01 เลือก Use Case จากงานประจำที่ทำซ้ำอย่างน้อยสัปดาห์ละครั้ง

2 นาที

งานที่นำเอามาที่สุดคือ Skill แรกที่คุ้มค่าที่สุด

02 เขียน Input / Process / Output ให้ครบก่อนลงมือ

6 นาที

เขียน Output ก่อน แล้วย้อนกลับมาเขียน Process

03 สร้าง Skill แล้วให้ Agent ทำงานตามโดยไม่ต้องอธิบายเพิ่ม

4 นาที

ถ้ายังต้องอธิบายเพิ่ม แปลว่ามีบรรทัดที่ยังขาด

04 ทดสอบซ้ำด้วยข้อมูลชุดใหม่ แล้วแก้ Skill จากผลที่ได้

3 นาที

ชุดใหม่คือชุดที่ Skill ยังไม่เคยเห็น

จบ 15 นาทีแล้วต้องมีไฟล์ SKILL.md หนึ่งไฟล์ ที่เรียกแล้วทำงานได้จริงอย่างน้อยหนึ่งรอบ

เลือก Use Case ใน 2 นาที

ตอบสามข้อนี้ในหัว ถ้าตอบได้ครบแปลว่าโจทย์ใช้ได้ เริ่มเขียนได้เลย

O1

งานอะไรที่เราทำซ้ำทุกสัปดาห์ แล้วไม่ค่อยอยากทำ

ตัวอย่าง · สรุปประชุม · เช็ค log เมื่อคืน · ทำรายงานยอด · ตอบอีเมลรูปแบบเดิม

O2

ถ้าเราสอนเพื่อนร่วมงานคนใหม่ จะบอกเป็นกี่ขั้น

ถ้าบอกได้ 3-6 ขั้น กำลังดี · ถ้าเกิน 10 ขั้น ให้ตัดเหลือเฉพาะส่วนที่ซ้ำจริง

O3

งานนี้ “เสร็จ” หน้าตาเป็นยังไง

ไฟล์อะไร เก็บไว้ที่ไหน มีหัวข้ออะไรบ้าง — ถ้าตอบไม่ได้ ยังเขียน Output ไม่ได้

ต้น 2 นาทีแล้วยังไม่ได้ ให้หยิบงานที่เพิ่งทำใน Workshop 5 มาทำเป็น Skill เลย — นั่นคือทางลัดที่ตั้งใจไว้

เทมเพลต · เติมช่องว่างแล้วใช้ได้เลย

เขียนให้จบใน 6 นาที แล้วค่อยแก้ — สั้นเกินแก้ง่ายกว่ายาวเกิน

HEADER

name: _____ - _____

description: ทำ _____
ให้ _____

ใช้เมื่อผู้ใช้พูดว่า “_____”

INPUT

ต้องมี _____ ก่อนเริ่ม

อยู่ที่ _____

ถ้าไม่มี ให้ _____ แทนการ
เดา

PROCESS

1. _____

2. _____

3. ถ้า _____ ให้ _____

4. ห้าม _____

OUTPUT

ไฟล์ _____ เก็บที่ _____

มีหัวข้อ _____ และ _____

ถือว่าเสร็จเมื่อ _____

ตัวอย่างที่เติมแล้ว

name: meeting-notes · ต้องมี บันทึกเสียงหรือนोटดิบ ก่อนเริ่ม · 3. ถ้าไม่มีผู้รับผิดชอบ ให้ถามก่อน · ไฟล์ notes/YYYY-MM-DD.md

ช่องว่างทุกช่องต้องเติมได้ด้วยคำที่ตรวจได้ — ถ้าเติมแล้วยังเถียงกันได้ว่าทำตามหรือยัง ให้เขียนใหม่

Hermes Agent Live Online · Dav 3

ทดสอบ Skill ของตัวเอง · สามรอบ

เขียนเสร็จยังไม่จบ — ผ่านสามรอบนี้แล้วค่อยเรียกว่าใช้งานได้

รอบที่ 1

เรียกตรง ๆ

บอกชื่อ Skill แล้วสั่งให้ทำ

✓ ทำครบทุกขั้นโดยเราไม่ต้องเสริมอะไรเลย

✗ ถ้ายังต้องเสริม แปลว่า Process ยังไม่ครบ

รอบที่ 2

เรียกด้วยภาษาคน

พิมพ์ประโยคที่จะพูดจริงในวันทำงาน ไม่บอกชื่อ Skill

✓ Agent หยิบ Skill ขึ้นมาเอง

✗ ถ้าไม่หยิบ ให้ไปแก้ description

รอบที่ 3

ป้อนข้อมูลไม่ครบ

จงใจไม่ให้ไฟล์ หรือให้ไฟล์ผิดรูปแบบ

✓ มันถามกลับ แทนที่จะเดาแล้วทำต่อ

✗ ถ้าเดา ให้เพิ่มเงื่อนไขในส่วน Input

รอบที่ 2 คือรอบที่คนตกมากที่สุด — Skill ที่เรียกไม่ติด เท่ากับไม่มี Skill

จาก Skill ส่วนตัว สู่ของที่ทีมใช้ร่วมกัน

จุดที่ Skill เริ่มคัมจริง คือตอนที่คนที่สองในทีมใช้มันโดยไม่ต้องถามเรา

- | | | |
|----|---|---|
| 01 | เก็บไว้ที่เดียวกับโค้ดหรือเอกสารของทีม | ไฟล์เดียว แก่ทีเดียว ทุกคนได้ของเวอร์ชันเดียวกัน |
| 02 | ให้เพื่อนลองใช้โดยที่เราไม่อธิบายอะไรเลย | ถ้าต้องอธิบาย แปลว่ายังไม่ใช่ Skill — เป็นแค่โน้ตของเรา |
| 03 | จุดจุดที่เพื่อนสะดุด แล้วแก้ที่ Skill ไม่ใช่บอกปากเปล่า | ทุกครั้งที่ต้องอธิบายซ้ำ คือหลักฐานว่ามีบรรทัดที่ยังขาด |
| 04 | ตั้งรอบทบทวนสั้น ๆ เดือนละครั้ง | ขั้นตอนของทีมเปลี่ยน Skill ที่ไม่ถูกแก้จะกลายเป็นของผิดภายในไม่กี่เดือน |

Skill คือความรู้ของทีมที่เขียนลงไฟล์ — คนลาออกได้ แต่วิธีทำงานยังอยู่

05

MCP

Mini Session · 21:55 – 22:00

MCP คืออะไร และสำคัญตรงไหน

Model Context Protocol คือมาตรฐานกลางสำหรับเชื่อม Agent เข้ากับ Tools และ Data Sources ภายนอก

01	ต่อเครื่องมือใหม่เข้ากับ Agent โดยไม่ต้องเขียนวิธีเชื่อมต่อเองทุกครั้ง	ต่อครั้งเดียว ใช้ได้กับ Agent ทุกตัวที่รองรับ
02	ระบบภายนอกประกาศความสามารถของตัวเองในรูปแบบเดียวกัน	Agent รู้เองว่าเครื่องมือไหนทำอะไรได้ โดยไม่ต้องสอน
03	เปลี่ยน Model หรือ Agent ได้ โดยเครื่องมือที่ต่อไว้ยังใช้ได้เหมือนเดิม	ย้ายไปโมเดลใหม่แล้วไม่ต้องรื้อระบบเชื่อมต่อ
04	เป็นพื้นฐานที่วันที่ 4 จะใช้ต่อ เมื่อ Agent เริ่มทำงานเป็นทีม	หลาย Agent แชร์เครื่องมือชุดเดียวกันได้

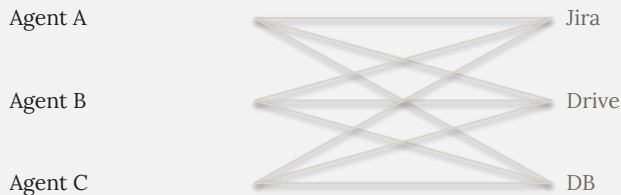
วันนี้รู้แล้วว่ามันคืออะไรก็พอ — วันที่ 4 กับ 5 เราจะต่อของจริงเข้ากับ Agent ของแต่ละคน

ทำไมต้องมีมาตรฐานกลาง

ปัญหาไม่ใช่ว่าต่อไม่ได้ — ต่อได้ แต่ต้องเขียนวิธีต่อใหม่ทุกครั้ง

ก่อนมี MCP

ต่อเองทุกคู่



$3 \times 3 = 9$ วิธีเชื่อมต่อที่ต้องดูแลเอง

มี MCP

ต่อผ่านมาตรฐานเดียว



$3 + 3 = 6$ การต่อ และเพิ่มของใหม่ได้โดยไม่กระทบของเดิม

เหมือนพอร์ต USB — ไม่ได้ทำให้อุปกรณ์เก้งขึ้น แต่ทำให้เสียบอะไรก็ได้โดยไม่ต้องหาสายเฉพาะรุ่น

MCP ใช้จริงหน้าตาเป็นแบบไหน

วันนี้ยังไม่ต้องลงมือ — แต่ให้เห็นภาพว่าวันที่ 4 กับ 5 เรากำลังจะไปไหน

งานเอกสาร

- ค้นไฟล์ใน Drive ของทีม
- อ่านสเปกแล้วสรุปให้
- เขียนสรุปกลับลงที่เดิม

งานพัฒนา

- อ่าน Issue และ Pull Request
- ดูสถานะ Build ล่าสุด
- เปิด Ticket ให้อัตโนมัติ

งานข้อมูล

- Query ฐานข้อมูลตามสิทธิ์
- ดึงตัวเลขมาทำรายงาน
- ไม่ต้อง export เป็น csv ก่อน

ข้อควรระวังที่ต้องพูดถึงเสมอ

ต่อ MCP หนึ่งตัว = ให้สิทธิ์เข้าถึงระบบจริงหนึ่งระบบ — ดูก่อนว่าใครเขียน ขอสิทธิ์อะไร และถอดออกได้ไหม
เริ่มจาก server ที่อ่านอย่างเดียว แล้วค่อยเปิดสิทธิ์เขียนเมื่อมั่นใจ

สรุปวันนี้

01

Agent ใช้เครื่องมือได้ และตรวจผลลัพธ์เองก่อนไปต่อ

สรุปเรียกเครื่องมือ · อ่านผล · ตัดสินใจต่อ

02

วิธีทำงานถูกเก็บเป็น Skill ที่เรียกใช้ซ้ำได้

Input / Process / Output ที่ตรวจได้

03

MCP เปิดทางให้ต่อระบบภายนอกตามมาตรฐานเดียวกัน

ต่อครั้งเดียว ใช้ได้หลาย Agent

04

การบ้าน · ใช้ Skill ของตัวเองกับงานจริงหนึ่งชิ้น

แก้ Skill จากผลจริง ไม่ใช่จากที่คิดไว้

ก่อน 16 ก.ย.

NEXT

Agent
Workflow &
Multi-Agent

16 ก.ย. 2569 · 20:00 น.

เอา Skill ของแต่ละคน
มาต่อกันเป็นสายงานเดียว

“Agent ที่ใช้เครื่องมือได้ คือ Agent ที่เริ่มทำงานแทนเราได้จริง”



Day 4 · Agent Workflow & Multi-Agent — 16 ก.ย. 2569, 20:00 น.

เอา Skill ที่เขียนวันนี้ไปใช้กับงานจริงหนึ่งชิ้น แล้วเจอกันวันที่ 4